Movie Recommendation and Tracker System:

Design and Analysis Report

Group 9

Group members:

Hongjun Gan 2471443

Yuhan Yan 2468817

Yiming Ye 2360720

Yishuo Zhang 2470897

Anwei Zou 2473081

1.System Design and Class Structure

The Movie Recommendation System is a modular Java application built on core object-oriented programming (OOP) principles, including encapsulation, modularity, and separation of concerns. This design ensures the system is maintainable, extensible, and adaptable to future enhancements. The architecture comprises nine interdependent core classes, each with well-defined responsibilities and interfaces.

1.1 OOP Principles in System Design

The system's design rigorously implements fundamental OOP principles to achieve code quality and flexibility:

(i) **Encapsulation**: The encapsulation principle is reflected in that all core classes of the system encapsulate data and their associated behaviors within a single unit: for instance, the User class packages user credentials (including username and encrypted password) as well as userspecific data like watchlist and history, and enables controlled access to these contents through getter methods; the Moviedatabase class conceals its internal storage implementation based on HashMap, only exposing public methods for data retrieval and manipulation; meanwhile, private fields (such as movieIds in the Watchlist class and movies in the Moviedatabase class) block direct external modification, thus ensuring the integrity of the data.

(ii) **Inheritance**: The core value of inheritance in OOP lies in code and behavior

reuse—subclasses inherit attributes and methods from superclasses to reduce redundancy, while supporting functional extension without altering parent class code. Though the system uses minimal explicit inheritance, Watchlist and History adopt a consistent core interface (such as addMovie (int movieId), contains (int movieId), getMovieIds()) to reuse identical collection management logic. Each class retains its unique purpose (managing favorites vs. viewing history), achieving both code reuse and functional differentiation.

(iii) **Modularity**: Following the modularity principle, the system is divided into independent, functionally focused modules that interact via well-defined interfaces: the authentication module (UserDatabase/User) handles user management and authentication; the movie data module (Moviedatabase/Movie) manages film data loading and querying; the user interaction module

(MovieApp/MovieGUI) coordinates workflows; and the recommendation module

(RecommendationEngine) implements algorithm logic independently, ensuring low coupling. (iv) **Polymorphism**: Polymorphism is realized through unified interfaces that adapt to different behaviors: the RecommendationEngine class offers multiple recommendation strategies (e.g., genre-based and diverse recommendations), which can be invoked uniformly via the getRecommendations() method; meanwhile, the parseIds() method can operate on both Watchlist and History objects by checking the target type, enabling a single method to handle different data collections.

1.2 Core Class Responsibilities

(a) **MovieApp**:

- **Primary Responsibilities**: Serves as the main application controller, managing user interaction through text-based menus and coordinating system operations. It handles the main program loop, user authentication flows, and movie management actions.
- **Key Methods**: start(), mainLoop(), showMainMenu(), showUserMenu().

(b) **UserDatabase**:

- **Primary Responsibilities**: Manages user data storage, authentication, and registration. It implements custom encryption for password security and handles CSV file operations for persistent storage of user information, including watchlists and viewing histories.
- **Key Methods**: loadUsers(), saveUsers(), authenticate(), register(), encrypt().

(c) **User**:

- **Primary Responsibilities**: Represents a system user with attributes for username, encrypted password, watchlist, and viewing history. It provides methods for converting watchlist and history data to CSV-compatible formats.
- **Key Methods**: getUsername(), getWatchlist(), getHistory(), watchlistToString().

(d) **Moviedatabase**:

- **Primary Responsibilities**:

Handles movie data management, including loading movies from CSV files, validating movie data, and providing access to movie information through various lookup methods.

- **Key Methods**: loadMovies(), getMovie(), getAllMovies(), getMoviesByGenre().

(e) **Movie**:

- **Primary Responsibilities**: Encapsulates movie attributes (ID, title, genre, year, rating) and provides a formatted string representation for display.
- **Key Methods**: getId(), getTitle(), toString().

(f) **RecommendationEngine**:

- **Primary Responsibilities**: Generates personalized movie recommendations using multiple strategies, including genre-based and diverse recommendations, without implementing explicit sorting algorithms.

- **Key Methods**: getRecommendations(), getGenreBasedRecommendations(), getDiverseRecommendations().

(g) **Watchlist**:

- **Primary Responsibilities**: Manages a collection of movie IDs representing films the user intends to watch (planned viewings). It supports full CRUD operations (create, read, update, delete) for movie entries, including adding new movies, removing unwanted entries, checking existence, and retrieving all saved movie IDs.
- **Key Methods**: addMovie(int movieId) removeMovie(int movieId) contains(int movieId) getMovieIds() (h) **History**:
- **Primary Responsibilities**: Tracks a collection of movie IDs representing films the user has already watched (completed viewings). It only supports write-and-read operations (no deletion) to maintain an immutable record of viewing activity, including adding watched movies, checking viewing history, and retrieving all watched movie IDs.
- **Key Methods**: addMovie(int movieId) contains(int movieId) getMovieIds() (i) **MovieGUI**:
- **Primary Responsibilities**: Responsible for graphical user interface interaction, undertaking the core logic of MovieApp and providing visual operation interfaces such as login, browsing, watchlist management, and recommendations. It supports mouse operations, table displays, and real-time alerts.
- **Key Methods**: start(), showLoginScreen(), showMainDashboard(), showBrowseView(), showRecommendationsView().

2. Recommendation Algorithm

2.1 Overall Recommendation Workflow

The RecommendationEngine.getRecommendations() method serves as the entry for the system's recommendation logic, following a "priority strategy" paradigm:

(a) First attempts genre-based recommendations (primary strategy).

(b) Automatically switches to diverse recommendations (fallback strategy) if the primary strategy yields no results (e.g., users with no viewing history or watchlist entries).

2.2 Detailed Core Recommendation Strategies

1. Genre-Based Recommendations (Primary Strategy)

This serves as the system's main recommendation strategy, following a three-step process:

Step 1: Analyze User Genre Preferences

(a) Iterates through the user's viewing history (History) and watchlist (Watchlist).

(b) Assigns weighted scores to each movie genre: +2 points for movies in the viewing history (higher weight for confirmed preferences) and +1 point for movies in the watchlist (lower weight for intended viewing).

(c) Data structure: Uses a HashMap<String, Integer> to store the genre-score mapping.

Step 2: Identify the User's Favorite Genre

(a)Finds the genre with the highest score through linear traversal (via the findFavoriteGenre() method), without using sorting algorithms.

(b)In case of a tie (multiple genres with the same highest score), the first encountered highestscoring genre is selected.

Step 3: Generate Recommendation Results

(a) Filters movies from the movie database that match the favorite genre.

(b) Excludes movies the user has already watched (in history) or added to their watchlist. (c) Retrieves the first N qualifying movies in their natural storage order (no sorting; directly takes the first N matches).

2. Diverse Recommendations (Fallback Strategy)

Activated when there is insufficient genre preference data for the user, aiming to provide movies from various genres:

Phase 1: Ensure Genre Diversity

(a) Iterates through the movie database, prioritizing movies from genres not yet selected (uses a

HashSet to track used genres).

(b) Also excludes movies the user has watched or added to their watchlist.

(c) Retrieves the first N movies from distinct genres.

Phase 2: Supplement Recommendations

(a) If the first phase fails to meet the required N recommendations, supplements with additional movies from the database.

(b) No genre restrictions during supplementation, but still excludes watched, watchlisted, and already recommended movies.

(c) Continues to take the remaining required number of movies in their natural order.

3. Year-Based Recommendations (Auxiliary Strategy)

This is an auxiliary recommendation interface (getRecommendationsByYear()) that requires explicit invocation:

(a) Filters movies by a specified release year.

(b) Excludes movies the user has watched or added to their watchlist.

(c) Retrieves the first N movies of the specified year in their natural storage order.

2.3 Algorithm Characteristics

1. **No sorting design**: All recommendation results are selected based on the natural storage order of movies in the database, without sorting by rating, popularity, or other factors.
2. **Simple rules**: Relies primarily on genre matching and basic filtering, with no complex machine learning models.
3. **Personalization**: Only uses the user's viewing history and watchlist as the basis for preference analysis.
4. **Real-time performance**: Recommendation results change in real time with user behaviors (viewing/adding to watchlist).

3. Testing Methods and Results

(1) UserAuthentication Module Testing Testing

Methods:

1. Test normal login flow: Log in with valid username and password.

2. Test failed login: Log in with incorrect username or password.
3. Test new user registration: Register with valid username and password.
4. Test duplicate user registration: Attempt to register with an existing username.
5. Test password change functionality: Verify login status changes before and after password modification.

Results:

1. Valid credentials login: Returns User object with successful login message.
2. Invalid credentials login: Returns null with error message displayed.
3. New user registration: Returns true, user is added to the database.
4. Duplicate user registration: Returns false with "username already exists" message.
5. Password change: Can log in with new password after successful modification; old password becomes invalid.

(2) Movie Management Module Testing Testing

Methods:

1. Test movie data loading: Verify correctness of movie loading from CSV file.
2. Test adding movies to watchlist: Use valid and invalid movie IDs.
3. Test removing movies from watchlist: Remove existing and non-existing movie IDs.
4. Test marking movies as watched: Verify movies are moved from watchlist to history.

Results:

1. Movie loading: Correctly parses CSV, skips invalid lines, and displays successful loading count.
2. Adding valid movie ID: Successfully added to watchlist with success message returned.
3. Adding invalid movie ID: Error message displayed, watchlist remains unchanged.
4. Removing existing movie: Successfully removed with success message returned.
5. Marking as watched: Movie is moved from watchlist to history (if present in watchlist).

(3) Recommendation Engine Testing Testing

Methods:

1. Test genre-based recommendations: When user has viewing history.
2. Test diverse recommendations: When user has no viewing history.

3. Test year-based recommendations: Specify a specific release year.

Results:

1.Genre-based recommendations: Prioritize movies of the user's favorite genre.

2.Diverse recommendations: Recommend movies of different genres.

3.Year-based recommendations: Return movies of the specified release year.

(4) GUI Module Testing Testing

Methods:

1. Test interface navigation: Switch between different views (browse/watchlist/history/ recommendations) via sidebar buttons.
2. Test right-click menu operations (watchlist): Mark as watched, remove from watchlist.
3. Test recommendation card operations: Add recommended movies to the watchlist.
4. Test alert prompts: Verify that operation results (success/error) are displayed via pop-up alerts.
5. Test window resizing: Verify that the interface layout adapts to window size changes.

Results:

1.Interface navigation: Smooth view switching with no data loss or lag.

2.Right-click operations: Accurate data modification and real-time CSV synchronization.

3.Recommendation card button: Triggers "Added!" state and valid watchlist update.

4.Pop-up prompts: Clear, scenario-matching messages with no ambiguity.

5.Window resizing: Dynamic layout adaptation with no overlapping or truncated elements.

4. Exception Handling and Input Validation

4.1 Exception Handling

(i)File Not Found Handling

1.Trigger Scenario: The specified movies.csv or users.csv file does not exist at the target path when loading data.

2.Source Code Examples:

(a) Movie File Not Found

```java
try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
    ...

} catch (FileNotFoundException e) {
    System.out.println(String.format("Error: Movie file not found: %s", filename));
    return false;
```

(b) User File Not Found

```java
try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
    ...
} catch (FileNotFoundException e) {
    System.out.println("Warning: Users file not found. Starting with empty database.");
```

3.Core Handling Mechanism

Catch FileNotFoundException and output a prompt with the missing path; launch an empty database to support registration if the user file is missing, or return a load failure flag if the movie file is missing, without blocking basic system operations.

(ii)I/O Exception Handling

1.Trigger Scenario: I/O errors occur during file read/write operations (e.g., corrupted files, insufficient disk permissions, file in use).

2.Source Code Examples

(a) Movie File Reading Exception

```java
try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
    ...
} catch (IOException e) {
    System.out.println(String.format("Error reading movie file: %s", e.getMessage()));
    return false;
}
```

(b) User File Saving Exception

```java
try (PrintWriter pw = new PrintWriter(new FileWriter(filename))) {
    ...
} catch (IOException e) {
    System.out.println(String.format("Error saving users: %s", e.getMessage()));
}
```

3.Core Handling Mechanism

Catch relevant exceptions separately and output targeted prompts; automatically close stream resources via try-with-resources; handle exceptions locally, return a result flag for failed read/write operations, and avoid impacting other modules.

(iii) Data Parse Failed Handling

1.Trigger Scenario: File reading succeeds, but CSV data does not meet requirements (e.g., missing fields, mismatched data types, values out of valid range).

2.Source Code Examples

(a) **Movie Data Parsing Exception**

```java
try {
    // Parse movie ID (handles M001 format)
    String idStr = parts[0].trim();
    int id;
    if (idStr.toUpperCase().startsWith("M")) {
        id = Integer.parseInt(idStr.substring( beginIndex: 1));
    } else {
        id = Integer.parseInt(idStr);
    }
    String title = parts[1].trim();
    String genre = parts[2].trim();
    int year = Integer.parseInt(parts[3].trim());
    double rating = Double.parseDouble(parts[4].trim());

    // Data validation
    if (year < 1880 || year > 2030) {
        System.out.println(String.format("Warning: Invalid year on line %d", lineNumber));
        continue;
    }
    if (rating < 0 || rating > 10) {
        System.out.println(String.format("Warning: Invalid rating on line %d", lineNumber));
        continue;
    }
    movies.put(id, new Movie(id, title, genre, year, rating));
    successCount++;

} catch (Exception e) {
    System.out.println(String.format("Warning: Invalid data on line %d: %s", lineNumber, e.getMessage()));
}
```

(b) Watchlist/History ID Parsing Exception

```java
private void parseIds(String str, Object targetList) {  2 usages
    if (str == null || str.isEmpty()) return;
    String[] items = str.split( regex: ";");
    for (String item : items) {
        try {
            String cleanId = item.trim();
            if (cleanId.contains("@")) cleanId = cleanId.split( regex: "@")[0];
            if (cleanId.toUpperCase().startsWith("M")) cleanId = cleanId.substring( beginIndex: 1);
            if (!cleanId.isEmpty()) {
                int id = Integer.parseInt(cleanId);
                ...
            }
        } catch (NumberFormatException e) {
            // Ignore invalid IDs
        }
    }
}
```

3.Core Handling Mechanism

Validate field legality; skip invalid rows with log warnings without affecting valid data loading. The password is only secured through encryption storage. When parsing, there is no need for decryption; it directly adapts to the storage format to ensure compatibility with the data.

(iv) Empty File Handling

1. Trigger Scenario: The loaded CSV file is empty (no content) or only contains headers without valid data lines.
2. Source Code Examples

(a) **Empty Movie File Handling**

```java
try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
    String line = br.readLine(); // Skip header

    int lineNumber = 1;
    int successCount = 0;

    while ((line = br.readLine()) != null) {
        lineNumber++;
        if (line.trim().isEmpty()) continue;

        ...
                successCount++;

            } catch (Exception e) {
                System.out.println(String.format("Warning: Invalid data on line %d: %s", lineNumber, e.getMessage()));
            }
        }
    }

    System.out.println(String.format(">>> Movies loaded: %d successful", successCount));
    return successCount > 0;

} catch (FileNotFoundException e) {
    System.out.println(String.format("Error: Movie file not found: %s", filename));
    return false;
} catch (IOException e) {
    System.out.println(String.format("Error reading movie file: %s", e.getMessage()));
    return false;
}
```

(b) **Empty User File Handling**

```java
try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
    String line = br.readLine(); // Skip header

    while ((line = br.readLine()) != null) {
        if (line.trim().isEmpty()) continue;

        ...
    }
} catch (FileNotFoundException e) {
    System.out.println("Warning: Users file not found. Starting with empty database.");
} catch (Exception e) {
    System.out.println(String.format("Error loading users: %s", e.getMessage()));
}
```

3. Core Handling Mechanism

Proactively detect empty data and prompt troubleshooting directions; return empty results for data-dependent functions without throwing exceptions.

4.2 Key Invalid Input Validation

1. Source Code Example

(a) **User Registration Validation**

```java
private void register() {  1 usage
    System.out.print("Choose a username: ");
    String username = scanner.nextLine().trim();
    System.out.print("Choose a password: ");
    String password = scanner.nextLine().trim();

    if (username.length() < 3 || password.length() < 4) {
        System.out.println("Error: Username > 3 chars, Password > 4 chars.");
        return;
    }
```

(b) **Movie ID Validation**

```java
private void addToWatchlist() {  1 usage
    System.out.print("Enter Movie ID: ");
    try {
        int id = Integer.parseInt(scanner.nextLine().trim());
        if (movieDB.exists(id)) {
            ...
        } else {
            System.out.println("Invalid Movie ID.");
        }
    } catch (NumberFormatException e) {
        System.out.println("Invalid input.");
    }
}
```

2. Core Validation Mechanism

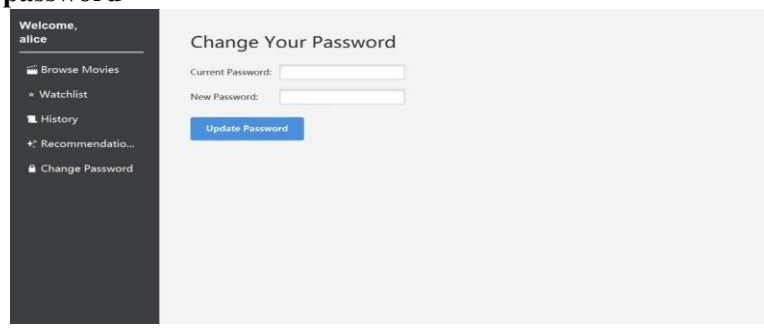
Adopt a multi-level process of "null check → format validation → range/existence validation"; output clear error prompts for different scenarios; immediately terminate subsequent logic if validation fails to avoid resource waste.

5. Example Screenshots of Program Operation

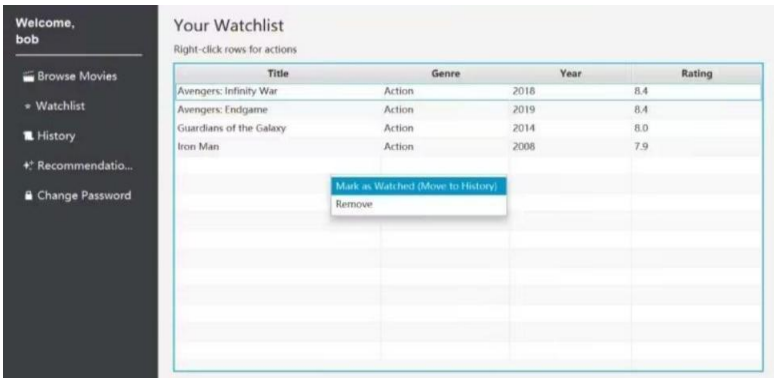
(1) **Create a new account**

(2) **Password recovery**

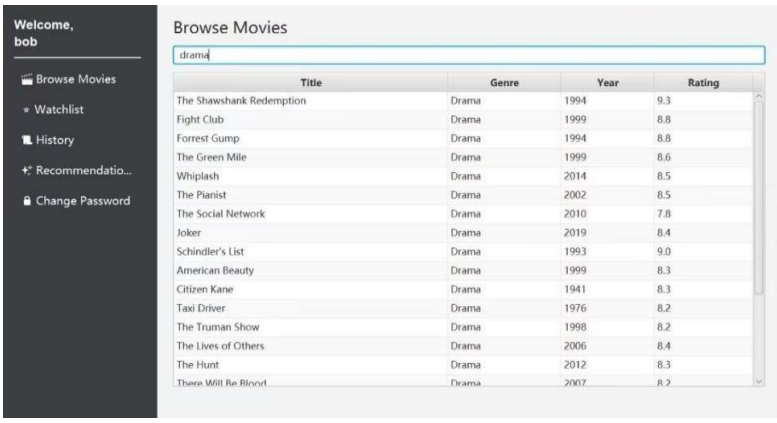
(3) **Change the password**



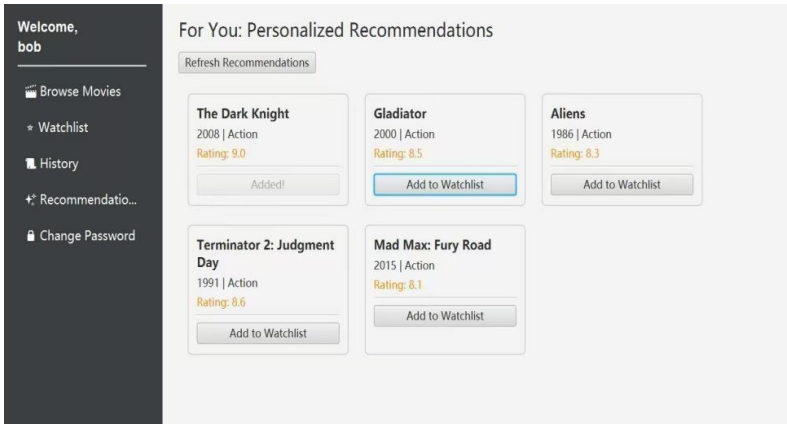
(4) **Watchlist**



(5) **Browse movies**

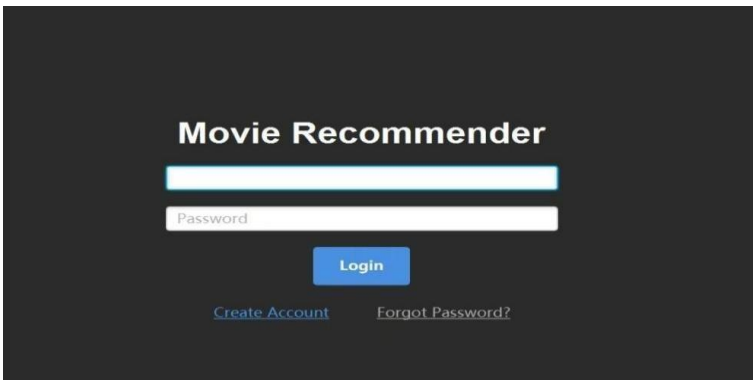


(6) **Personalized recommendations**



(7) **Viewing history**

(8) **Recommender**



6. Explanation of How to Apply Software Engineering Team Management Practices to Complete This Project

As a team, we are applying structured software engineering management practices to ensure the successful completion of this movie recommendation system project. Below is how we organize and manage our development process:

(1) **Role Assignment**

We divided six people into a coding group and a reporting group. Some people are responsible for implementing the functionality of the code, while others are responsible for testing, statistics, and generating the final report. Among the developers responsible for coding, we further divided different individuals into different classes to lighten each person's workload.

(2) **Iterative Development**

The project is developed in iterative cycles. We started with basic functionality like user login and movie browsing, then gradually added features like watchlist management, history tracking, and

recommendation algorithms. Each iteration includes design, implementation, testing, and review phases.

(3) **Object-Oriented Design Approach**

Following the specification, we apply OOP principles by designing core classes such as User, Movie, Watchlist, History, and RecommendationEngine. Inheritance and encapsulation are used where appropri- ate, e.g., subclassing User into BasicUser and PremiumUser.

(4) **Continuous Integration and Testing**

Unit tests are written for critical functions such as loading/saving data from/to CSV files (UserDataMan- ager, MovieDatabase) and generating recommendations. Exception handling is implemented throughout the application to avoid crashes. Regular integration tests ensure components work together seamlessly.

(5) Documentation and Communication

Clear documentation is maintained for class designs, method specifications, and user manuals. We hold regular stand-up meetings to discuss progress, challenges, and next steps. Tools like Slack and Trello aid real-time communication and task tracking.

7. Ethical Evaluation of the System and Solutions to Potential Problems

(1) **Privacy Protection Issues**

- User Data Collection Risks: The system stores users' viewing history (History), watchlist (Watchlist), and other personal preference information, posing a risk of privacy leakage.
- Password Security Issues: In the current implementation, user passwords (Password) are stored in plain text in CSV files, making them vulnerable to unauthorized access.

(2) **Risks of Recommendation System Abuse**

- Algorithm Bias: The recommendation algorithm (RecommendationEngine) based on user viewing history may lead to the "filter bubble" effect, limiting users' exposure to diverse content.
- Over-Personalization: Over-reliance on user behavior data may result in overly narrow recommen- dation outcomes, affecting the diversity of user experience.

(3) **User Rights Protection**

- Provide data export and deletion functions, allowing users to manage their History and Watchlist data.
- Increase transparency regarding the recommendation algorithm, helping users understand the logic behind recommendations.
- Implement explainability for recommendation results to help users understand why a particular movie is recommended.